

Guide de dépannage

Jegonflable Aube — du seed Prisma au déploiement Vercel

Ce document récapitule les **7 erreurs** les plus courantes rencontrées entre l'installation locale et la mise en production sur Vercel, avec pour chacune : la cause, le diagnostic et le fix.

Profil cible : stagiaire qui découvre Next.js + Prisma + Vercel.

Pré-requis : Node 20+, npm, un compte Neon, un compte GitHub, un compte Vercel.

Sommaire

- 1. **Erreur seed** — *Environment variable not found: DATABASE_URL*
- 2. **Prisma + .env.local** — pourquoi ça marche pour Next.js mais pas pour Prisma
- 3. **Erreur EPERM** sur *prisma generate* (Windows)
- 4. **Git config** — *key does not contain a section: global*
- 5. **Git push** — *src refspec main does not match any*
- 6. **Build Vercel KO** — *Failed to collect page data for /api/auth/[...nextauth]*
- 7. **Build Vercel KO #2** — *Prisma Client outdated on Vercel*
- **Annexe** — checklist de déploiement Vercel

1. Erreur seed Prisma

Symptôme

```
Erreur seed : PrismaClientInitializationError:  
Invalid `prisma.user.upsert()` invocation in prisma/seed.ts:22  
  
error: Environment variable not found: DATABASE_URL.  
--> schema.prisma:10
```

Cause

Prisma cherche la variable `DATABASE_URL` au lancement du seed et ne la trouve nulle part. Le piège : Prisma CLI lit le fichier `.env` à la racine du projet, **pas** `.env.local`.

Fix

Renommer `.env.local` en `.env` (Next.js charge les deux, donc l'app continue de fonctionner).

```
# Dans VS Code : clic droit sur .env.local > Renommer > .env  
# Ou en ligne de commande :  
mv .env.local .env
```

Puis vérifier que ton fichier `.env` contient au minimum :

```
DATABASE_URL="postgresql://user:pwd@ep-xxx.neon.tech/db?sslmode=require"  
NEXTAUTH_SECRET="dev-secret-change-en-prod"  
NEXTAUTH_URL="http://localhost:3000"
```

■ **Note** — Sur Windows, l'Explorateur cache les extensions par défaut. Vérifie que le fichier s'appelle bien `.env` et pas `.env.txt` avec `dir /a` dans le terminal.

2. Prisma + .env.local : pourquoi ça coince

Petit tableau pour ne plus jamais se faire avoir :

Outil	Lit .env	Lit .env.local
next dev / next build	✓	✓
prisma generate / db push / migrate	✓	✗
tsx prisma/seed.ts	✗ (sans dotenv)	✗
npm run db:seed (via Prisma)	✓	✗

✓ Règle : pour ce projet, **un seul fichier .env** à la racine. Tout le monde est content.

3. Erreur EPERM sur prisma generate (Windows)

Symptôme

```
EPERM: operation not permitted, rename
'node_modules/.prisma/client/query_engine-windows.dll.node.tmp...'
-> 'node_modules/.prisma/client/query_engine-windows.dll.node'
```

Cause

Le fichier `query_engine-windows.dll.node` est verrouillé par un autre processus — typiquement `next dev` qui tourne encore dans un terminal voisin, Prisma Studio ouvert, OneDrive en train de synchroniser, ou Windows Defender qui scanne.

Fix

Étape 1 — fermer tout ce qui touche au projet :

- Arrêter `next dev` (Ctrl+C)
- Fermer Prisma Studio s'il est ouvert
- Arrêter tout script `npm run db:seed` en cours

Étape 2 — tuer les processus Node zombies :

```
taskkill /F /IM node.exe
```

Étape 3 — relancer :

```
npx prisma generate
```

Si ça persiste, supprimer le dossier généré et recommencer :

```
Remove-Item -Recurse -Force node_modules\.prisma
npx prisma generate
```

■ **Note** — Bonne pratique : ne pas mettre les projets dev dans un dossier synchronisé OneDrive (Bureau, Documents). Préférer `C:\dev\` ou similaire — moins de locks.

4. Git config — "key does not contain a section: global"

Symptôme

```
$ git config global user.name "Maxence Bueb"  
error: key does not contain a section: global
```

Cause

Faute de frappe : il manque les **deux tirets** devant `global`. Git interprète `global` comme une clé de configuration (au format `section.cle`) et trouve qu'elle n'a pas de section, d'où l'erreur.

Fix

```
git config --global user.name "Maxence Bueb"  
git config --global user.email "buebmaxence@gmail.com"
```

Vérification :

```
git config --global --list
```

Tu dois voir tes deux lignes `user.name` et `user.email`.

5. Git push — "src refs spec main does not match any"

Symptôme

```
$ git push -u origin main  
error: src refspec main does not match any  
error: failed to push some refs to 'https://github.com/.../...git'
```

Cause

Trois pistes possibles :

- **Mauvais dossier** : ton terminal n'est pas dans `jegonflable-aube/`.
- **Aucun commit** n'a encore été créé : la branche `main` n'existe que *après* le premier commit.
- **Ta branche locale s'appelle master** (ancien défaut Git) et tu pousses vers `main`.

Diagnostic

```
cd jegonflable-aube # se mettre dans le bon dossier  
git status # voir les fichiers à commit  
git branch # nom de la branche actuelle  
git log --oneline # historique des commits
```

Fix selon le cas

Cas A — pas de commit encore :

```
git add .  
git commit -m "Initial commit"  
git push -u origin main
```

Cas B — branche locale `master` :

```
git branch -m master main  
git push -u origin main
```

6. Build Vercel KO — Failed to collect page data

Symptôme

```
> Build error occurred
[Error: Failed to collect page data for /api/auth/[...nextauth]]
Error: Command "npm run build" exited with 1
```

Cause

Pendant le build, Next.js importe la route `/api/auth/[...nextauth]`, qui importe `lib/auth.ts`, qui importe `lib/db.ts`, qui instancie `PrismaClient`. Sans `DATABASE_URL` côté Vercel, l'init crashe.

Important : Vercel ne lit pas ton `.env` local — il est gitignoré, donc absent du dépôt distant. Il faut déclarer chaque variable explicitement dans le dashboard Vercel.

Fix

Sur `vercel.com` → ton projet → **Settings** → **Environment Variables**, ajouter ces 3 variables (cocher Production, Preview et Development) :

Variable	Valeur
DATABASE_URL	ta connection string Neon (la même que dans <code>.env</code>)
NEXTAUTH_SECRET	<code>openssl rand -base64 32</code> (ou n'importe quelle longue string)
NEXTAUTH_URL	<code>https://<ton-projet>.vercel.app</code>

Puis aller dans l'onglet **Deployments** → cliquer sur `...` du déploiement raté → **Redeploy**.

Variables additionnelles selon les features utilisées : `BREVO_SMTP_*`, `EMAIL_FROM`, `ADMIN_EMAIL` pour l'email ; `STRIPE_*` si Stripe est activé ; `NEXT_PUBLIC_SITE_URL`, `NEXT_PUBLIC_SITE_NAME`.

7. Build Vercel KO #2 — Prisma Client outdated

Symptôme

```
Error [PrismaClientInitializationError]: Prisma has detected that
this project was built on Vercel, which caches dependencies.
This leads to an outdated Prisma Client because Prisma's auto-generation
isn't triggered. To fix this, make sure to run the prisma generate
command during the build process.
```

Cause

Vercel met en cache `node_modules` entre les builds. `@prisma/client` est conservé tel quel, mais le code généré par `prisma generate` (qui dépend de `schema.prisma`) n'est pas régénéré. Résultat : le client utilisé ne correspond pas au schéma → crash.

Fix recommandé

Ajouter un script `postinstall` dans `package.json` qui lance `prisma generate` automatiquement après chaque `npm install` :

```
"scripts": {
  "postinstall": "prisma generate",
  "build": "next build",
  ...
}
```

Cette solution couvre **Vercel** (regenère à chaque build) et **localement** (regenère après chaque `npm install` qui suit un changement de schéma).

Alternative

Modifier directement le script de build :

```
"build": "prisma generate && next build"
```

Moins propre, mais ça marche aussi. Une fois le `package.json` modifié, `commit + push`, Vercel rebuild automatiquement.

Annexe — checklist de déploiement Vercel

À suivre dans l'ordre pour un premier déploiement propre, après que l'app tourne en local.

Avant de pousser

- `npm run build` passe en local (sinon ça passera pas non plus sur Vercel)
- `.env` contient toutes les variables nécessaires
- `package.json` a bien le `postinstall: prisma generate`
- `.gitignore` exclut bien `.env` et `node_modules`
- Tous les changements sont commités et poussés sur GitHub

Sur Vercel

- **Import** le repo GitHub depuis le dashboard Vercel
- **Framework Preset** : Next.js (auto-déecté)
- **Environment Variables** : ajouter *toutes* celles du `.env` (`DATABASE_URL`, `NEXTAUTH_SECRET`, `NEXTAUTH_URL`, `BREVO_*`, etc.)
- **Déployer** et attendre le build

Après le déploiement

- Aller sur l'URL `https://<projet>.vercel.app`
- Vérifier que la page d'accueil charge (sinon → logs Vercel)
- Tester le login (avec l'admin créé par le seed)
- Si le seed n'a pas tourné en prod : ouvrir Prisma Studio en local sur la même DB Neon, ou lancer `npx prisma db seed` avec la `DATABASE_URL` de prod en variable d'env

Pour debugger un build Vercel qui plante

- Onglet **Deployments** → cliquer sur le déploiement raté → **Build Logs**
- Le vrai message d'erreur est *au-dessus* du Command `"npm run build"` exited with 1
- Tester en local avec `npm run build` avant de re-pousser
- Vérifier que *toutes* les variables d'env sont bien présentes et orthographiées sans typo

Document généré pour le stagiaire — Maxime / WEBOMAX. Couvre les erreurs rencontrées entre le seed Prisma et le déploiement Vercel.